

SYSTEM REQUIREMENTS SPECIFICATION

INTELLIX

The Academic Operating System

Architecture: Laravel 11 · React · Rust · Gemini AI

Version: 1.0.0

Status: Draft for Development

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Product Vision / Business Case	3
1.2.1	The Problem Space	3
1.2.2	The Solution	3
1.3	Scope	4
1.3.1	Core Modules (In Scope)	4
1.3.2	Target Audience	4
1.4	Out of Scope (for Version 1.0)	4
1.5	Success Metrics	4
1.6	Definitions and Acronyms	5
1.7	Retrieval-Augmented Generation (RAG) Model	5
2	System Architecture	7
2.1	High-Level Design	7
2.2	Component Responsibilities	8
2.2.1	The Orchestrator (Laravel 11)	8
2.2.2	The Execution Units (Rust Binaries)	8
2.2.3	The Persistence Layer	8
2.3	Data Pipelines	8
2.3.1	Pipeline A: The Ingestion Flow (Upload)	8
2.3.2	Pipeline B: The RAG Query Flow (Chat)	9
2.4	Architectural Considerations / Constraints	9
2.4.1	1. Cross-Platform Compilation (The "It Works on My Machine" Problem)	9
2.4.2	2. Observability / Debugging	9
2.4.3	3. Resource Contention (The "Thundering Herd")	9
2.4.4	4. Vector Database Scalability	10
3	Functional Requirements (Sprint Backlog)	11
3.1	Overview	11
3.2	Epic 1: Data Ingestion Pipeline	11
3.3	Epic 2: The "Brain" (RAG System)	12
3.4	Epic 3: Active Learning Tools	13
3.5	Epic 4: Productivity Suite	14
4	Data Model Specification	15
4.1	Entity Relationship Diagram (ERD)	15
4.2	Storage Strategy	16

5	Interface Specifications	18
5.1	Internal CLI Contract (Laravel → Rust)	18
5.1.1	Web Scraper Interface	18
5.1.2	PDF Processor Interface	19
5.2	External API Specification (REST + SSE)	20
5.2.1	Ingestion Endpoints	20
5.2.2	RAG Chat Endpoint (Streaming)	20
5.2.3	3. Study FSRs Endpoint	20
6	Quality Attributes (Non-Functional Requirements)	21
6.1	Performance Metrics (SLA)	21
6.2	Security Compliance	21
6.2.1	1. Data Isolation (Row-Level Security)	21
6.2.2	2. Input Sanitization	21
6.3	Reliability Fault Tolerance	22
6.3.1	1. Circuit Breaker Pattern	22
6.3.2	2. Idempotency	22
6.4	Scalability Strategy	22
6.4.1	1. Horizontal Scaling (Worker Nodes)	22
6.4.2	2. Database Sharding (Future Proofing)	22
7	Project Cost Analysis & Infrastructure	23
7.1	Overview	23
7.2	Infrastructure Strategy (The "Bootstrap" Stack)	23
7.2.1	1. Compute (VPS)	23
7.2.2	2. Storage (Self-Hosted MinIO)	23
7.2.3	3. AI Consumption (Gemini 1.5 Flash)	24
7.3	Monthly Cost Breakdown (MVP Phase)	24
7.4	Scaling Triggers Future Costs	24
7.5	Conclusion on Feasibility	25
8	Conclusion Future Outlook	26
8.1	Executive Summary	26
8.2	Technical Feasibility	26
8.3	Strategic Value Impact	26
8.4	Roadmap to Launch	27

Chapter 1

Introduction

1.1 Purpose

The purpose of this document is to define the functional and non-functional requirements for **Intellix**, an AI-native Academic Operating System.

Intellix is designed to address the "Crisis of Attention" in modern universities by consolidating the fragmented academic workflow—note-taking, file management, flashcard creation, and exam preparation—into a single, unified platform. Unlike generic AI tools, Intellix uses a **Retrieval-Augmented Generation (RAG)** architecture to ensure all AI outputs are strictly grounded in the user's provided course materials, prioritizing accuracy over generative creativity.

1.2 Product Vision Business Case

1.2.1 The Problem Space

University students today face three critical challenges:

1. **The Trust Gap:** Generic LLMs (like ChatGPT) frequently "hallucinate" facts, making them unreliable for serious academic work where precision is mandatory.
2. **Passive Learning:** Students often "hoard" digital notes without actively engaging with them, leading to poor retention and exam performance.
3. **Fragmented Workflows:** Technical students (STEM) struggle with tools that don't support code or LaTeX, while Humanities students struggle with tools that lack citation management.

1.2.2 The Solution

Intellix solves these by providing a ****Grounded Knowledge Engine****:

- **Accuracy First:** AI responses are generated *only* from uploaded PDFs and notes, with inline citations to the source text.
- **Active Recall:** The system automatically converts passive notes into active Spaced Repetition (SRS) flashcards.
- **Universal Format:** A rich-text editor that supports both Code Blocks/LaTeX (for CS/Engineering) and Long-form Citation/Essay drafting (for Law/Humanities).

1.3 Scope

The Intellix system encompasses the full data lifecycle of a university student:

1.3.1 Core Modules (In Scope)

- **Ingestion Layer (Rust/Laravel):** High-performance parsing of diverse external data including Web URLs, PDF Text/Images, and Lecture Audio.
- **Knowledge Graph (PostgreSQL/Vector):** Storing semantic relationships between topics, notes, and generated insights.
- **Study Engine (React):** Interactive tools including Spaced Repetition (SRS) flashcards, Code Snippet execution, and Exam Simulation.
- **Output Generator (AI):** Automated creation of reports, essays, and slide decks based *strictly* on user-provided context.

1.3.2 Target Audience

The platform is designed to support multidisciplinary requirements:

- **STEM Users:** Requires syntax highlighting, LaTeX equation rendering ($E = mc^2$), and diagram support.
- **Humanities Users:** Requires robust citation management (APA/MLA styles) and semantic search across large text corpuses.

1.4 Out of Scope (for Version 1.0)

To ensure a timely MVP release, the following features are explicitly excluded:

- **Real-Time Collaboration:** Simultaneous multi-user editing (e.g., Google Docs style) is deferred.
- **Native Mobile Applications:** V1.0 will be a Responsive Web Application (PWA); native iOS/Android apps are out of scope.
- **Note Marketplace:** Peer-to-peer selling of notes is planned for Version 2.0.
- **LMS Integration:** Direct integration with university systems (Canvas/Blackboard) is out of scope.

1.5 Success Metrics

The success of Intellix will be measured by **User Outcome** metrics rather than vanity metrics:

1. **Retention Rate:** 40% of active users engage with the Flashcard (SRS) system daily.
2. **Study Efficiency:** Users report a 50% reduction in time spent organizing/searching for notes.
3. **Accuracy Trust:** AI Summaries achieve a <5% hallucination rate (verified by citation checking).

1.6 Definitions and Acronyms

RAG	Retrieval-Augmented Generation: The architectural pattern of retrieving relevant document chunks from the database to ground the LLM's response.
SRS	Spaced Repetition System: An algorithmic approach (based on SM-2/FSRS) to schedule reviews at optimal intervals to combat the "Forgetting Curve."
Vector Embeddings	High-dimensional numerical representations of text allowing for semantic search (e.g., matching "coding" to "programming").
Hallucination	A phenomenon where an LLM generates plausible-sounding but factually incorrect information.

1.7 Retrieval-Augmented Generation (RAG) Model

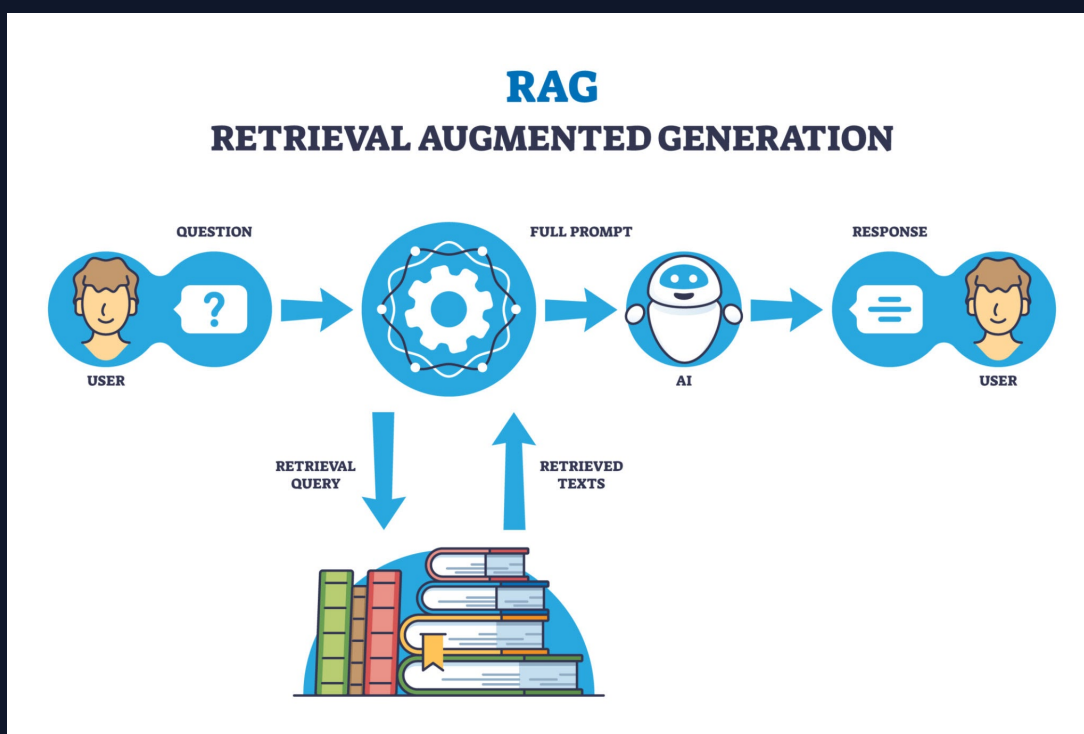


Figure 1.1: Overview of the RAG Architecture

Process Overview

The Retrieval-Augmented Generation (RAG) model is a powerful AI framework designed to improve the quality and accuracy of large language models (LLMs). It combines pre-trained LLMs with an external knowledge retrieval component to generate more informative and factual responses. This approach allows the model to access up-to-date information and reduce the likelihood of generating incorrect or made-up answers.

Workflow Steps

1. User's Question

The process begins with a user posing a question to the system. This initial query is

the catalyst for the entire RAG workflow. The question represents the user's need for information, which the model will attempt to fulfill.

2. Retrieval Query

The user's question is then transformed into a retrieval query. This query is used to search a vast knowledge base, typically a large collection of documents or a database, for relevant information. The goal is to find text passages that are most likely to contain the answer to the user's question.

3. Knowledge Base and Retrieved Texts

The retrieval system scours the knowledge base and extracts the most relevant texts. These retrieved texts serve as a rich source of factual information for the LLM. The quality and relevance of these texts are crucial for the final response's accuracy.

4. Full Prompt and AI

The retrieved texts are then combined with the original user question to form a "full prompt." This augmented prompt is then fed into the AI, which is a pre-trained large language model. The AI now has both the user's query and relevant contextual information to work with.

5. Generated Response

Finally, the AI generates a comprehensive and informative response. Because the response is grounded in the retrieved texts, it is more likely to be accurate, detailed, and up-to-date. This final answer is then presented back to the user, completing the RAG cycle.

Chapter 2

System Architecture

2.1 High-Level Design

Intellix utilizes a **Modular Monolith** architecture designed for high throughput and maintainability.

The system is strictly divided into three layers:

- **Presentation Layer (React):** A "thin client" handling UI state and optimistic updates.
- **Orchestration Layer (Laravel):** The authoritative API gateway that manages authentication, business rules, and job dispatching.
- **Execution Layer (Rust):** Ephemeral, high-performance binaries invoked for CPU-intensive tasks.

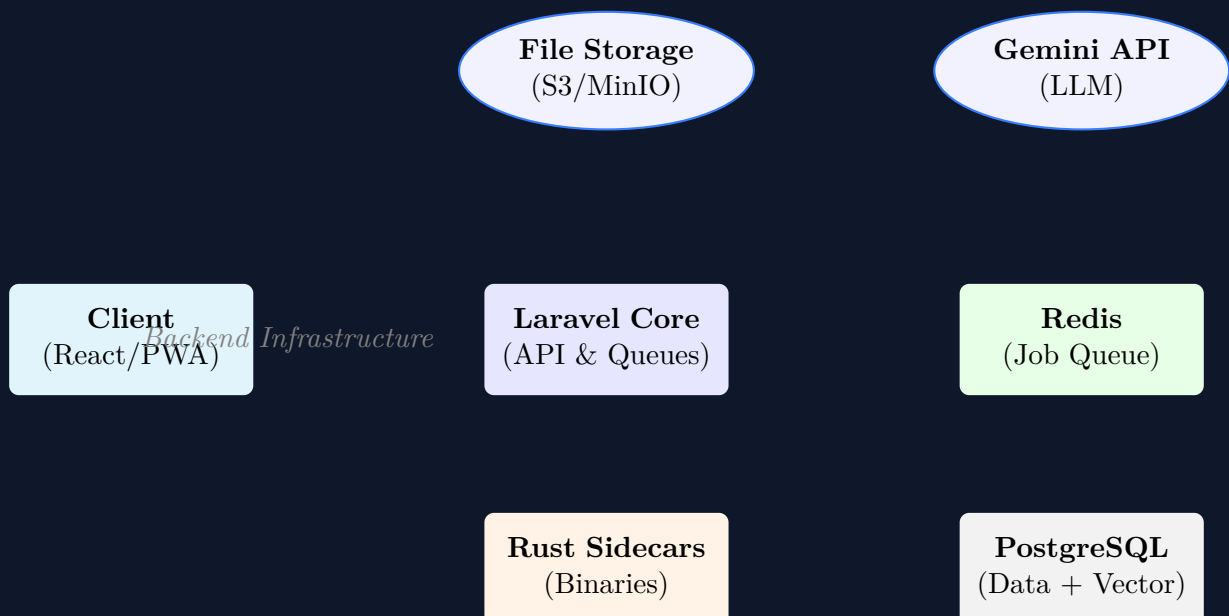


Figure 2.1: System Architecture & Communication Flows

2.2 Component Responsibilities

2.2.1 The Orchestrator (Laravel 11)

Laravel serves as the central nervous system. Crucially, it handles the **Asynchronous Job Pipeline**:

- **API Gateway:** Validates requests (Sanctum) and rate-limits users.
- **Queue Worker:** Listens to Redis for resource-heavy tasks (e.g., `ProcessPdfJob`).
- **Prompt Engineering:** Dynamically constructs prompts for Gemini based on user context before sending the request.

2.2.2 The Execution Units (Rust Binaries)

Rust binaries are stateless and ephemeral. They communicate with Laravel via the **Standard Streams (stdio)** contract:

- **Input:** Laravel passes file paths and options via Command Line Arguments.
- **Output:** Rust prints a structured JSON object to `STDOUT`.
- **Error Handling:** Critical failures are printed to `STDERR` with a non-zero exit code, which Laravel catches and logs.

2.2.3 The Persistence Layer

Data storage is split to optimize for cost and speed:

- **Hot Data (PostgreSQL):** User profiles, Note text, and Vector Embeddings ('pgvector').
- **Cold Data (Blob Storage):** Original PDF files and audio recordings are stored in an S3-compatible object store (e.g., MinIO for local dev, AWS S3 for prod) to keep the database lean.

2.3 Data Pipelines

2.3.1 Pipeline A: The Ingestion Flow (Upload)

This pipeline converts raw files into queryable knowledge.

1. **Upload:** User uploads a PDF via React.
2. **Storage:** Laravel streams the file to S3 and returns a `202 Accepted` response.
3. **Queue:** A `ProcessFile` job is pushed to Redis.
4. **Processing:** The Queue Worker executes the `pdf-processor` Rust binary.
5. **Vectorization:** Laravel chunks the returned text and sends it to Gemini Embedding API.
6. **Persistence:** Vectors are stored in PostgreSQL.

2.3.2 Pipeline B: The RAG Query Flow (Chat)

This pipeline answers user questions with high accuracy.

1. **Embed:** User query is converted to a vector (Gemini).
2. **Search:** PostgreSQL finds the nearest 3 vectors using Cosine Similarity ($\leq$ operator).
3. **Synthesize:** Laravel sends the Query + 3 Context Chunks to Gemini 1.5 Flash.
4. **Response:** The AI generates an answer with citations.

2.4 Architectural Considerations Constraints

While the Modular Monolith architecture offers significant benefits in terms of code organization and type safety, it introduces specific operational challenges that must be managed.

2.4.1 1. Cross-Platform Compilation (The "It Works on My Machine" Problem)

Risk: The development environment (Windows/macOS) differs from the production environment (Linux). Rust binaries compiled on the host machine will fail to execute on the server due to ABI (Application Binary Interface) incompatibilities.

Mitigation Strategy:

- **Containerization:** The entire application stack is containerized using Docker. The `Dockerfile` employs a multi-stage build process where the Rust binary is compiled within a `rust:alpine` container before being copied to the final PHP image.
- **Targeted Compilation:** For local testing without Docker, developers must use the `x86_64-unknown-linux-musl` target to generate statically linked binaries compatible with any Linux distribution.

2.4.2 2. Observability Debugging

Risk: The communication between Laravel and Rust occurs via standard streams (STDIN/STDOUT). If a Rust binary panics or fails silently, the PHP application may hang or receive an empty response, leading to a "Black Box" debugging scenario.

Mitigation Strategy:

- **Structured Logging:** All Rust binaries must implement a strict logging contract. Errors are never printed as raw text but as structured JSON objects to `STDERR`, which Laravel captures and logs to the application monitoring system (Telescope/Sentry).
- **Exit Codes:** The binaries must return specific exit codes (e.g., `exit(1)` for network failure, `exit(2)` for parse error) to allow Laravel to retry the job intelligently.

2.4.3 3. Resource Contention (The "Thundering Herd")

Risk: Spawning a new OS process for every user request is computationally expensive. A sudden spike in traffic could lead to process exhaustion, crashing the server.

Mitigation Strategy:

- **Queue Throttling:** Resource-intensive tasks (PDF parsing, Scraping) are never executed synchronously. They are pushed to a Redis Queue with a strictly limited number of worker processes (e.g., `-max-jobs=2`), ensuring the server CPU is never overwhelmed.

2.4.4 4. Vector Database Scalability

Risk: Storing high-dimensional vectors (1536 dimensions for Gemini) requires significant RAM for the HNSW index. As the dataset grows, the index size may exceed available memory, causing severe performance degradation.

Mitigation Strategy:

- **Index Quantization:** We will utilize `pgvector`'s support for half-precision (float16) indexing to reduce memory usage by 50% with negligible impact on retrieval accuracy.
- **Selective Indexing:** Only the "Cleaned Text" is vectorized; raw HTML and metadata are stored in standard columns, keeping the vector index lean.

Chapter 3

Functional Requirements (Sprint Backlog)

3.1 Overview

This chapter decomposes the high-level features into atomic, implementable tasks. **Priority Levels:**

- **P1 (Core):** Must be completed in the first 3 Sprints.
- **P2 (Beta):** Required for the initial public release.
- **P3 (Enhancement):** Polish and optimizations.

3.2 Epic 1: Data Ingestion Pipeline

Goal: Allow users to upload files and URLs, transforming them into clean text.

Task ID	Actionable Requirement	Pri
Feature: File Upload (Laravel)		
ING-01.1	API Endpoint - File Upload Create <code>POST /api/upload</code> that accepts PDF/Audio. Validate max size (50MB) and mime types.	P1
ING-01.2	S3 Storage Integration Stream uploaded files to S3 (or MinIO dev) to avoid memory spikes. Return a unique <code>file_key</code> .	P1
ING-01.3	Database Record Creation Create a row in <code>resources</code> table with status <code>'processing'</code> and reference the S3 key.	P1
Feature: PDF Processor (Rust Binary)		
ING-02.1	CLI Argument Parsing Rust binary <code>pdf-processor</code> must accept <code>-input-path</code> and <code>-output-format=json</code> .	P1

Task ID	Actionable Requirement	Pri
ING-02.2	Text Extraction Logic Use <code>lopdf</code> crate to extract raw text strings from PDF pages.	P1
ING-02.3	Heuristic Cleaning Remove recurring artifacts (e.g., page numbers "Page 1 of 20", footers) from extracted text.	P2
ING-02.4	JSON Output Contract Output must match schema: <code>{ "pages": [{"page": 1, "text": "..."}] }</code> .	P1
Feature: Web Scraper (Rust Binary)		
ING-03.1	HTTP Fetcher Rust binary <code>scraper-cli</code> fetches HTML from a URL with a rotating User-Agent to avoid 403 blocks.	P1
ING-03.2	DOM Sanitization Use <code>scraper</code> crate to remove <code><script></code> , <code><style></code> , <code><nav></code> , and <code><footer></code> tags.	P1
ING-03.3	Markdown Converter Convert remaining HTML (h1, p, ul, table) into standard Markdown format.	P1
Feature: Orchestration (Laravel Queue)		
ING-04.1	ProcessFileJob Create a Laravel Job that runs the specific Rust binary based on file type.	P1
ING-04.2	Failure Handling If Rust returns non-zero exit code, update <code>resources.status</code> to 'failed' and log <code>STDERR</code> .	P1

3.3 Epic 2: The "Brain" (RAG System)

Goal: Make the uploaded text searchable and usable by AI.

Task ID	Actionable Requirement	Pri
Feature: Vectorization		
RAG-01.1	Chunking Logic Split cleaned text into overlapping segments (Size: 512 tokens, Overlap: 50 tokens).	P1
RAG-01.2	Embedding Job Send batches of chunks to Gemini Embedding API. Handle rate limits (429 Too Many Requests).	P1

Task ID	Actionable Requirement	Pri
RAG-01.3	pgvector Storage Insert vectors into <code>resource_vectors</code> table. Ensure <code>resource_id</code> foreign key is set.	P1
Feature: AI Chat API		
RAG-02.1	Search Service Implement Cosine Similarity query: <code>ORDER BY embedding <=> query_vector LIMIT 5</code> .	P1
RAG-02.2	Context Injection Construct System Prompt: "You are a tutor. Answer ONLY using these context snippets: [...]".	P1
RAG-02.3	Streaming Response Implement Server-Sent Events (SSE) to stream the AI answer character-by-character to the frontend.	P2
RAG-02.4	Citation Append Append source links (e.g., [Page 5]) to the answer based on which chunks were used.	P2

3.4 Epic 3: Active Learning Tools

Goal: Convert passive notes into active exams and flashcards.

Task ID	Actionable Requirement	Pri
Feature: Flashcard Core		
STU-01.1	AI Generator Prompt Design prompt: "Extract 5 key concepts from this text and format as JSON: {front: '...', back: '...'}".	P1
STU-01.2	Bulk Insert Save generated cards to <code>flashcards</code> table. Default <code>interval = 0</code> .	P1
STU-01.3	Review UI Endpoint <code>GET /api/decks/{id}/review</code> : Return only cards where <code>next_review < NOW()</code> .	P1
Feature: FSRS Algorithm (Rust)		
STU-02.1	Rust Calculation Struct Implement the FSRS formula in Rust. Input: (<code>last_interval</code> , <code>difficulty</code> , <code>rating</code>). Output: <code>new_interval</code> .	P2

Task ID	Actionable Requirement	Pri
STU-02.2	Batch Update Job Daily Job: Call Rust binary to recalculate 'stability' for all cards reviewed yesterday.	P2
Feature: Mock Exams		
STU-03.1	Question Generator AI Prompt to generate distractors for Multiple Choice Questions (A, B, C, D).	P2
STU-03.2	Exam State Management React Frontend: Track user answers array. Prevent page refresh loss (Local Storage).	P2

3.5 Epic 4: Productivity Suite

Goal: Planning and creating output deliverables.

Task ID	Actionable Requirement	Pri
Feature: Study Planner		
PRO-01.1	Calendar UI React Component: Drag-and-drop calendar (using <code>react-big-calendar</code>).	P3
PRO-01.2	Constraints Logic Calculate "Pages per Day" required to finish a deck by the exam date.	P3
Feature: Presentation Builder		
PRO-02.1	Outline Generation AI Prompt: "Create a 10-slide outline for a presentation on [Topic]".	P3
PRO-02.2	Slide Content Expansion For each slide title, generate 3 bullet points and a speaker note.	P3
PRO-02.3	PDF Compiler (Rust) Rust binary uses <code>printpdf</code> to draw text/shapes onto a canvas and export <code>slides.pdf</code> .	P3

Chapter 4

Data Model Specification

4.1 Entity Relationship Diagram (ERD)

The Intellix data architecture is split into three distinct domains: **Identity**, **Knowledge Ingestion (RAG)**, and **Active Study (SRS)**. The following diagram illustrates how unstructured data (PDFs) flows into structured vectors and eventually into study artifacts (Flashcards).

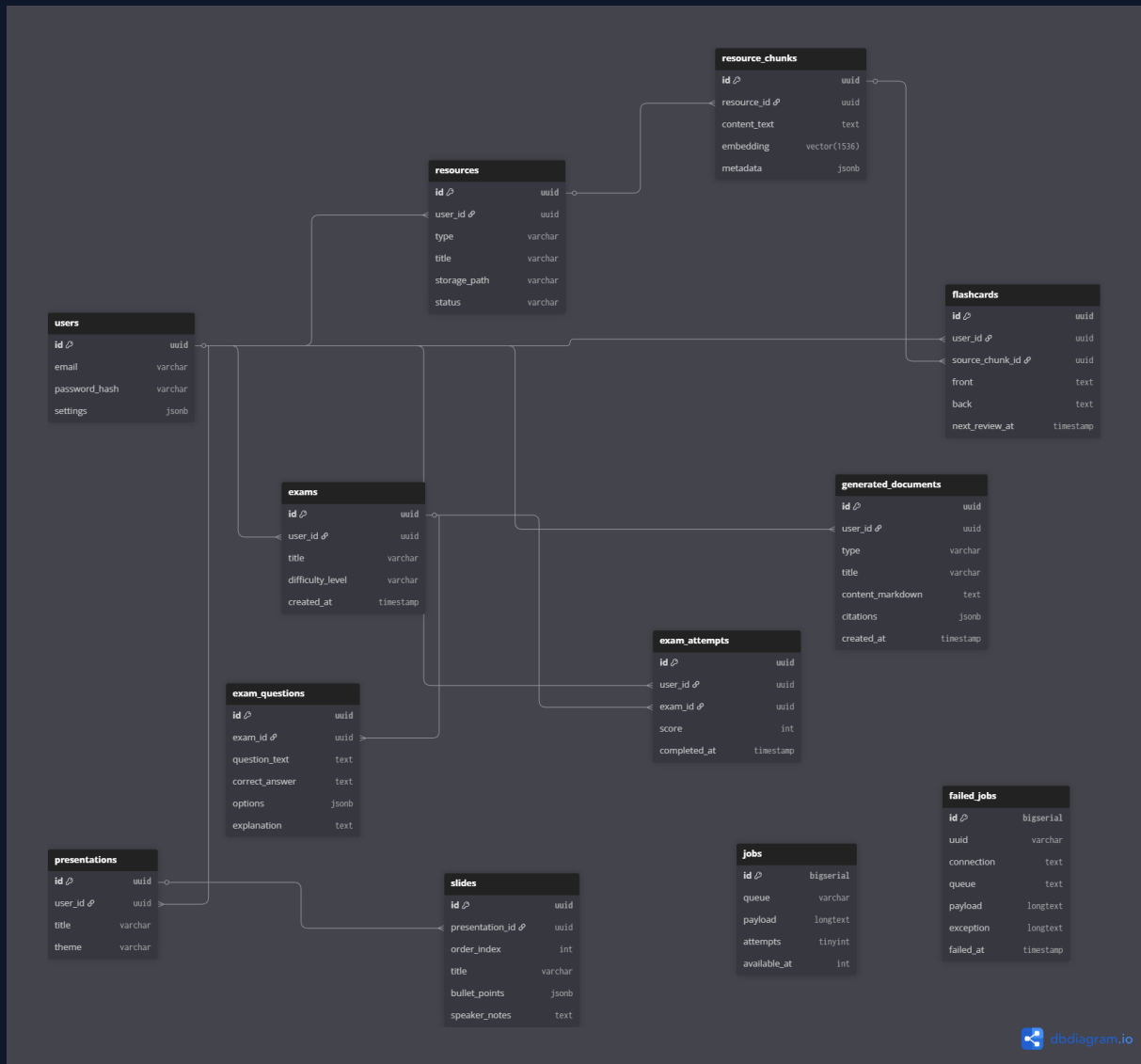


Figure 4.1: Intellix v1.0 Database Schema: RAG & SRS Pipelines

4.2 Storage Strategy

To optimize performance and costs, Intellix employs a **Tiered Storage Strategy**:

- **Tier 1: Hot Storage (PostgreSQL)**
 - Stores textual metadata, flashcards, and vector embeddings.
 - **Constraint:** `content_text` in 'resource_chunks' must be kept under 2KB per row to prevent bloating the database pages (TOAST).
- **Tier 2: Object Storage (S3 / MinIO)**
 - Stores the original immutable binaries (PDFs, Audio files).
 - **Path Structure:** `/uploads/{user_id}/{resource_id}.pdf`
 - This ensures the database backup remains small (MBs) even if users upload GBs of files.
- **Tier 3: Ephemeral Storage (Redis)**

- Stores job queues ('processing_pdf') and temporary chat history sessions.
- Acts as a cache for frequent RAG queries to save Gemini API costs.

Chapter 5

Interface Specifications

5.1 Internal CLI Contract (Laravel → Rust)

The backend orchestrates Rust binaries via standard streams (STDIN/STDOUT). All binaries must output strictly formatted JSON.

5.1.1 Web Scraper Interface

Binary: bin/scraper-cli

Command:

```
1 ./bin/scraper-cli --url="https://arxiv.org/abs/2106.09685" --timeout=30
```

Success Output (STDOUT):

```
1 {
2   "status": "success",
3   "metadata": {
4     "title": "LoRA: Low-Rank Adaptation of Large Language Models",
5     "author": "Edward J. Hu et al.",
6     "published_at": "2021-06-17"
7   },
8   "content_markdown": "# LoRA: Low-Rank Adaptation...\n\nWe propose..."
9 }
```

Error Output (STDERR):

```
1 {
2   "status": "error",
3   "code": "TIMEOUT_REACHED",
4   "message": "Target host did not respond within 30s."
5 }
```

5.1.2 PDF Processor Interface

Binary: bin/pdf-processor

Command:

```
1 ./bin/pdf-processor --source="/storage/app/uploads/tmp/8f92a.pdf"
```

Success Output (STDOUT):

```
1 {
2   "status": "success",
3   "total_pages": 12,
4   "chunks": [
5     { "page": 1, "text": "Introduction..." },
6     { "page": 1, "text": "Background Theory..." },
7     { "page": 2, "text": "Methodology..." }
8   ]
9 }
```

5.2 External API Specification (REST + SSE)

The frontend communicates via stateless REST APIs. Authentication is handled via Laravel Sanctum (Bearer Token).

5.2.1 Ingestion Endpoints

POST /api/v1/resources/upload

Handles multipart file uploads. Large files are streamed directly to S3.

- **Content-Type:** multipart/form-data
- **Body:** file (Binary), folder_id (UUID)
- **Response (202 Accepted):**

```
1 {
2   "message": "File queued for processing",
3   "resource_id": "550e8400-e29b-41d4-a716-446655440000",
4   "status_url": "/api/v1/resources/550e8400.../status"
5 }
```

5.2.2 RAG Chat Endpoint (Streaming)

POST /api/v1/chat/completions

Uses Server-Sent Events (SSE) to stream the AI response character-by-character.

- **Body:**

```
1 {
2   "query": "Explain the difference between TCP and UDP",
3   "context_filter": { "folder_id": "123" },
4   "mode": "strict" // 'strict' = citation required
5 }
```

Streamed Response Events:

```
1 data: {"chunk": "TCP is connection-oriented"}
2 data: {"chunk": " whereas UDP is connectionless."}
3 data: {"citation": {"source_id": "8f92a...", "page": 4}}
4 event: stop
```

5.2.3 3. Study FSRS Endpoint

POST /api/v1/flashcards/id/review

Submits a review for the Spaced Repetition algorithm (FSRS).

- **Body:**

```
1 {
2   "rating": 3,
3   // 1=Again (Fail), 2=Hard, 3=Good, 4=Easy
4   "duration_ms": 4500
5   // Time taken to answer (affects 'difficulty' score)
6 }
```

Response (200 OK):

```
1 {
2   "next_review": "2026-02-14T10:00:00Z",
3   "interval_days": 4,
4   "stability": 2.5
5 }
```

Chapter 6

Quality Attributes (Non-Functional Requirements)

6.1 Performance Metrics (SLA)

The system must adhere to the following strict latency budgets to ensure a fluid user experience.

Operation	Max Latency	Conditions
Static Asset Load	< 100ms	CDN Hit (Cloudflare/AWS CloudFront)
Vector Search	< 200ms	Dataset < 1M vectors, HNSW Index Active
AI First Token (TTFB)	< 1.5s	Time to First Byte for streaming chat response
PDF Ingestion	< 5s	Per 10 pages of standard text content

6.2 Security Compliance

6.2.1 1. Data Isolation (Row-Level Security)

Requirement: Multi-tenancy must be enforced at the database query level.

- **Implementation:** All RAG queries (vector searches) must include a `WHERE user_id = ?` clause.
- **Verification:** Automated tests must attempt to query User A's vectors with User B's token and assert a 403 Forbidden or empty result.

6.2.2 2. Input Sanitization

Requirement: Prevention of XSS (Cross-Site Scripting) and Injection attacks.

- **Scraping:** All HTML fetched by Rust binaries must pass through an allow-list sanitizer (e.g., ammonia crate) stripping `<script>`, `<iframe>`, and `onclick` attributes before storage.

- **LaTeX Injection:** User-provided math equations must be rendered in a sandboxed iframe or sanitized to prevent remote code execution via LaTeX macros.

6.3 Reliability Fault Tolerance

6.3.1 1. Circuit Breaker Pattern

Since the system relies on external APIs (Gemini, Whisper), it must handle downstream failures gracefully.

- **Threshold:** If the AI API fails 5 times in 1 minute.
- **Action:** The system enters "Open State," serving cached responses or a friendly "Service Busy" message for 60 seconds before retrying.
- **Implementation:** Laravel Guzzle Middleware or Redis-backed circuit breaker.

6.3.2 2. Idempotency

Requirement: Critical operations (e.g., billing, file processing) must handle duplicate requests safely.

- **Job Queue:** Redis jobs must be unique by `resource_id`. If a user uploads the same PDF twice, the second job should detect the existing hash and skip processing.

6.4 Scalability Strategy

6.4.1 1. Horizontal Scaling (Worker Nodes)

The **Modular Monolith** architecture allows for independent scaling of the "Muscle" (Rust) and the "Brain" (Laravel).

- **Scenario:** CPU usage spikes due to heavy PDF processing during exam week.
- **Response:** Auto-scaling group spins up 5 additional "Worker Droplets" containing only the Rust binaries and Queue Worker. The main web server remains unaffected.

6.4.2 2. Database Sharding (Future Proofing)

While the MVP uses a single PostgreSQL instance:

- The `users` and `resources` tables are designed with UUIDs to facilitate future horizontal sharding (e.g., sharding by `user_id`) without primary key collisions.

Chapter 7

Project Cost Analysis & Infrastructure

7.1 Overview

This chapter outlines the financial feasibility and infrastructure strategy for Intellix. The priority is to minimize Operational Expenditure (OpEx) by leveraging self-hosted open-source solutions (MinIO, PostgreSQL) on bare-metal Virtual Private Servers (VPS).

7.2 Infrastructure Strategy (The "Bootstrap" Stack)

To maintain a near-zero burn rate during the initial launch phase, Intellix will avoid expensive managed services (AWS RDS, S3, Vercel) in favor of a containerized "Box-in-a-Box" architecture.

7.2.1 1. Compute (VPS)

Provider: Hetzner Cloud (Falkenstein, Germany) or DigitalOcean (London).

Reasoning: Low latency to West Africa (< 120ms) and best price-to-performance ratio.

- **Spec:** 2 vCPU (ARM64/AMD), 4GB RAM, 40GB NVMe.
- **OS:** Ubuntu 24.04 LTS (Dockerized).
- **Role:** Hosts the entire stack: Laravel (App), Postgres (DB), Redis (Queue), and MinIO (Storage).

7.2.2 2. Storage (Self-Hosted MinIO)

Instead of paying for AWS S3 Egress fees, we will deploy **MinIO** as a Docker container on the same VPS.

- **Capacity:** 40GB (System) + Block Storage Volume (scalable).
- **Backup Strategy:** Daily incremental snapshots of the Docker Volume to an external cold storage (e.g., Backblaze B2) for disaster recovery.
- **Cost Impact:** \$0.00/mo (uses existing VPS resources).

7.2.3 3. AI Consumption (Gemini 1.5 Flash)

We utilize the ****Google AI Studio Free Tier**** for the MVP phase.

- **Limit:** 15 Requests Per Minute (RPM), 1,500 Requests Per Day.
- **Usage Strategy:** Heavy caching of summaries ensures we stay within limits for the first 500 daily active users.

7.3 Monthly Cost Breakdown (MVP Phase)

The following budget assumes a single VPS instance capable of handling $\approx 1,000$ users.

Component	Service / Configuration	Monthly Cost
Compute (VPS)	Hetzner CPX21 (3 vCPU / 4GB RAM)	\$8.50
Object Storage	MinIO (Self-Hosted Docker)	\$0.00
Database	PostgreSQL 16 (Self-Hosted)	\$0.00
AI Inference	Gemini 1.5 Flash (Free Tier)	\$0.00
Domain Name	Namecheap (.app / .io)	\$1.20
SSL Certificates	Let's Encrypt (Automated)	\$0.00
Email Delivery	Brevo (300 emails/day free)	\$0.00
TOTAL OPEX	Est. Monthly Burn Rate	\$9.70

7.4 Scaling Triggers Future Costs

As the user base grows, specific bottlenecks will trigger infrastructure upgrades.

Trigger	Action	New Cost Est.
Storage Full (> 80%)	Attach 100GB Block Storage Volume to VPS.	+\$5.00 / mo
AI Rate Limit (> 15 RPM)	Switch to Gemini Paid Tier (\$0.35/1M tokens).	Variable ($\approx$ \$10)
RAM Exhaustion	Move Database to Managed Postgres Instance.	+\$15.00 / mo

7.5 Conclusion on Feasibility

With a projected operating cost of under **\$10.00 per month**, Intellix is highly viable as a student-led project. The architecture allows for "vertical scaling" (upgrading the VPS) up to approx. 5,000 users before requiring complex architectural changes (Kubernetes/Load Balancers).

Chapter 8

Conclusion Future Outlook

8.1 Executive Summary

This document has outlined the technical and functional specifications for **Intellix**, an AI-native Academic Operating System designed to solve the "fragmentation crisis" in modern education.

By moving beyond simple note-taking and integrating a **Retrieval-Augmented Generation (RAG)** engine, Intellix offers a unique value proposition: it is not just a storage bin for files, but an active study partner that "reads" and "understands" course material alongside the student.

8.2 Technical Feasibility

The proposed **Modular Monolith** architecture (Laravel + React + Rust) represents a pragmatic balance between development speed and raw performance:

- **Stability:** Laravel provides a battle-tested foundation for authentication, queuing, and API management.
- **Performance:** Offloading CPU-intensive tasks (PDF parsing, Web scraping) to compiled **Rust binaries** ensures the application remains responsive even under heavy load.
- **Cost Efficiency:** By leveraging self-hosted infrastructure (MinIO, PostgreSQL) and the Gemini Free Tier, the project achieves a monthly burn rate of < \$10.00, making it highly viable for a student-led launch.

8.3 Strategic Value Impact

Intellix addresses the "Trust Gap" that currently prevents universities from fully embracing AI. Unlike generic tools like ChatGPT, which often hallucinate, Intellix's **Citation-First** approach builds trust.

For the end-user (the student), the impact is measurable:

- **Reduced Anxiety:** Automated planning and exam simulations replace "cramming."
- **Active Retention:** The automated conversion of passive notes into active Spaced Repetition (SRS) cards forces long-term memory formation.
- **Time Savings:** The ability to "chat" with a 500-page textbook reduces research time by hours per week.

8.4 Roadmap to Launch

The immediate path forward is clear and actionable:

1. **Sprint 1 (Weeks 1-2):** Initialize the Laravel repository and build the `scraper-cli` Rust binary.
2. **Sprint 2 (Weeks 3-4):** Deploy the MVP with basic PDF upload and "Chat with Document" features.
3. **Sprint 3 (Weeks 5-6):** Implement the Flashcard FSRS algorithm and invite the first 50 Beta testers.

"The future of education is not just about access to information, but the intelligent synthesis of it. Intellix is the tool that makes that future accessible today."